

AES variants and their comparisons

Sreejit Maity

6th August 2026

1 Introduction

The Advanced Encryption Standard (AES) is a symmetric-key block cipher standardized by the National Institute of Standards and Technology (NIST). Utilizing a fixed block size of 128 bits (16 bytes), AES processes state transformations through a substitution-permutation network. The cipher operates via three discrete variants distinguished strictly by key length: 128 bits, 192 bits, and 256 bits.

2 Structural Characteristics and Differences

The internal security threshold of each AES variant is mathematically bound to its key space. To prevent structural shortcut attacks across extended keys, the Rijndael design linearly scales the iteration space (N_r) alongside the key schedule requirements (N_k).

- **AES-128:** Implements a 4-word key ($N_k = 4$) driving 10 mathematical transformation rounds. It handles a combinatorial brute-force search space of 2^{128} .
- **AES-192:** Implements a 6-word key ($N_k = 6$) driving 12 mathematical transformation rounds. It expands the complexity margin to 2^{192} .
- **AES-256:** Implements an 8-word key ($N_k = 8$) driving 14 mathematical transformation rounds. This provides a 2^{256} quantum-resistant security ceiling.

Except for the terminal round which bypasses the `MixColumns` phase, each algorithmic round alters the internal state matrix via sequential byte-level transformations:

$$\text{State} \rightarrow \text{SubBytes} \rightarrow \text{ShiftRows} \rightarrow \text{MixColumns} \rightarrow \text{AddRoundKey} \quad (1)$$

3 C Implementations and Benchmarking Framework

To quantify hardware-level execution differentials, the single C implementation below embeds a low-level `rdtsc` (Read Time-Stamp Counter) hardware instruction to precisely count elapsed CPU clock cycles.

```
1 #include <stdio.h>
2 #include <stdint.h>
3 #include <string.h>
4
5 #define BLOCK_SIZE 16
6
7 // Inline hardware hook to track true CPU Clock Cycles via Assembly
8 static inline uint64_t get_clock_cycles() {
9     unsigned int lo, hi;
10     __asm__ __volatile__ ("rdtsc" : "=a" (lo), "=d" (hi));
```

```

11     return ((uint64_t)hi << 32) | lo;
12 }
13
14 // Educational Byte Substitution Matrix (S-Box Configuration)
15 static const uint8_t sbox[64] = {
16     0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0
17     xfe, 0xd7, 0xab, 0x76,
18     0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0
19     x9c, 0xa4, 0x72, 0xc0,
20     0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0
21     x71, 0xd8, 0x31, 0x15,
22     0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0
23     xeb, 0x27, 0xb2, 0x75
24 };
25
26 void sub_bytes(uint8_t *state) {
27     for (int i = 0; i < BLOCK_SIZE; i++) {
28         state[i] = sbox[state[i] % 64];
29     }
30 }
31
32 void shift_rows(uint8_t *state) {
33     uint8_t temp = state[1];
34     state[1] = state[5]; state[5] = state[9]; state[9] = state[13]; state[13] =
35     temp;
36 }
37
38 void mix_columns(uint8_t *state) {
39     for (int i = 0; i < BLOCK_SIZE; i++) {
40         state[i] = (state[i] << 1) ^ (state[i] >> 7 ? 0x1B : 0x00);
41     }
42 }
43
44 void add_round_key(uint8_t *state, const uint8_t *round_key) {
45     for (int i = 0; i < BLOCK_SIZE; i++) {
46         state[i] ^= round_key[i];
47     }
48 }
49
50 void execute_aes(const uint8_t *input, uint8_t *output, const uint8_t *key, int
51     key_bits, uint64_t *cycles) {
52     int rounds = (key_bits == 128) ? 10 : ((key_bits == 192) ? 12 : 14);
53     uint8_t state[BLOCK_SIZE];
54     memcpy(state, input, BLOCK_SIZE);
55
56     uint64_t start = get_clock_cycles();
57
58     add_round_key(state, key);
59     for (int r = 1; r < rounds; r++) {
60         sub_bytes(state);
61         shift_rows(state);
62         mix_columns(state);
63         add_round_key(state, key);
64     }
65     sub_bytes(state);
66     shift_rows(state);
67     add_round_key(state, key);
68
69     uint64_t end = get_clock_cycles();
70     *cycles = end - start;
71     memcpy(output, state, BLOCK_SIZE);
72 }

```

```

68 int main() {
69     uint8_t plaintext[BLOCK_SIZE] = {0x32, 0x43, 0xf6, 0xa8, 0x88, 0x5a, 0x30,
    0x8d, 0x31, 0x31, 0x98, 0xa2, 0xe0, 0x37, 0x07, 0x34};
70     uint8_t ciphertext[BLOCK_SIZE];
71
72     uint8_t key_128[16] = {0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae, 0xd2, 0xa6, 0xab
    , 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c};
73     uint8_t key_192[24] = {0x8e, 0x73, 0xb0, 0xf7, 0xda, 0x0e, 0x64, 0x52, 0xc8
    , 0x10, 0xf3, 0x2b, 0x80, 0x90, 0x79, 0xe5, 0x62, 0xf8, 0xea, 0xd2, 0x52, 0
    x2c, 0x6b, 0x7b};
74     uint8_t key_256[32] = {0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca, 0x71, 0xbe, 0x2b
    , 0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81, 0x1f, 0x35, 0x2c, 0x07, 0x3b, 0
    x61, 0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4};
75
76     uint64_t c_128, c_192, c_256;
77
78     execute_aes(plaintext, ciphertext, key_128, 128, &c_128);
79     execute_aes(plaintext, ciphertext, key_192, 192, &c_192);
80     execute_aes(plaintext, ciphertext, key_256, 256, &c_256);
81
82     printf("AES-128 Clock Cycles: %llu\n", (unsigned long long)c_128);
83     printf("AES-192 Clock Cycles: %llu\n", (unsigned long long)c_192);
84     printf("AES-256 Clock Cycles: %llu\n", (unsigned long long)c_256);
85
86     return 0;
87 }

```

Listing 1: Single C Program for Structural AES Benchmarking

4 Performance Comparisons and Cycle Metrics

Because the structural architecture of the execution loop expands with the key size, CPU requirements scale metrics proportionally. Software-driven AES engines register a near-linear expansion in CPU cycle requirements matching the loop parameters.

Table 1: Cryptographic and Performance Metrics Comparison

AES	Key Size	Rounds (N_r)	Security	Relative Performance
AES-128	128 bits	10	High (2^{128})	Baseline ($1.0 \times$ Cycles)
AES-192	192 bits	12	Very High (2^{192})	Moderate Overhead ($\approx 1.2 \times$)
AES-256	256 bits	14	Maximum (2^{256})	Maximum Overhead ($\approx 1.4 \times$)

5 Conclusion

Selecting the optimal AES variant requires evaluating the explicit trade-off between strict run-time performance constraints and localized data lifespan parameters. For high-throughput edge systems or processing pipeline structures, AES-128 minimizes CPU cycle consumption. Conversely, for critical infrastructures or institutional assets exposed to long-term structural decrypt threats, the 40% computational cycle increase required by AES-256 is an essential investment to guarantee robust security.

I have taken the help of various websites, Wikipedia and Gemini for smoothly compiling the codes, details and Latex orientations.